Assignment 2: Text Processing and Classification using Apache Spark (100p)

Filip Aleksic, Lucija Aleksic, Anabel Dautovic, Ana Petrovic, Matyas Prechl

1. Introduction

In this assignment, we explore the full potential of Apache Spark by implementing a scalable text classification pipeline using both RDD-based and DataFrame-based approaches. The task is structured in three parts, each designed to highlight different aspects of Spark's ecosystem - from low-level RDD transformations to high-level MLlib Pipelines. By combining classic information retrieval techniques such as TF-IDF and chi-square feature selection with modern machine learning approaches like Support Vector Machines (SVMs), the assignment aims to build a robust model that can predict product categories from user reviews. Additionally, this work emphasizes the importance of reproducibility, parameter tuning, and comparative analysis in machine learning workflows.

2. Problem Overview

The goal of the assignment is to classify product reviews into predefined categories based solely on the review text. The problem is approached in three stages:

1. **RDD-Based Feature Extraction**: We manually tokenize, preprocess, and compute chi-square statistics using Spark's RDD API. This step simulates low-level distributed data manipulation and results in a ranked list of most informative terms per category.

2. **DataFrame-Based Pipeline Construction**: A structured Spark ML pipeline is constructed using built-in transformers such as `RegexTokenizer`, `StopWordsRemover`, `CountVectorizer`, `IDF`, and `ChiSqSelector`. This pipeline automates the TF-IDF transformation and feature selection, preparing the data for downstream classification.

3. **Text Classification with SVM**: The pipeline is extended with a `OneVsRest` classification model using a `LinearSVC` as the base classifier. The model is trained and evaluated on a train/validation/test split using F1 score. We conduct grid search to optimize hyperparameters such as regularization strength, standardization, and number of iterations, and we compare the performance of models trained on 2000 vs. 500 top chi-square-selected features.

Through this three-part process, we compare various abstraction levels of Spark, evaluate different feature sets and model parameters, and gain hands-on experience in building scalable machine learning pipelines for text data.

3. Methodology and Approach

Part 1

The goal of Part 1 is to replicate the first assignment, this time with an additional level of abstraction through the use of Spark RDDs (Resilient Distributed Datasets) and transformations on them. In the context of Spark, working with RDDs is a very low-level approach and it is not recommended for real world applications and deprecated.

We can load the dataset directly from HDFS into an RDD and apply our transformations to this. For efficient processing we will run only one mapper function on this RDD and the further transformations will be applied to the resulting RDD.
This mapper function is `category_token_doc_count_mapper` in the code. We still need to filter, map and reduce this RDD to produce the data we need for the chi-square calculations.
We produce:

- A total document count as a global variable
- A document count per category as an RDD
- A document count per token as an RDD
- And a document count per token within a category as an RDD

(Similarly to as described in our report for Assignment 1 - Section 3)

Then we define mapper function for calculating the chi-square values, which will be applied to the last RDD (`category_token_doc_counts`), while the other 2 RDDs we broadcast (we make them available on all nodes, so the appropriate values are available for the calculation. An alternative to these broadcasts would be to make all the necessary values for the calculation available within the key-value pair in the RDD to which we apply the calculation mapper function, which could improve performance.

The calculation is done exactly the same way as described in our report for Assignment 1 - Section 4.

Once the chi-square values are calculated we can apply a few more sorting, grouping and formatting transformations to produce the desired output, which we write to a file.

Part 2

For Part 2, the goal was to build a Spark-based pipeline to process the Amazon review dataset, convert the review texts into a vector space representation, and prepare the data for classification.

Comparison of Assignment 1 and assignment output_ds:
Example overlapping terms from Assignment 1 and Part 2 of Assignment 2:
561. Around ⅓ of all words from Assignment 1 appear in the 2000 words from Assignment 2 Part 2.

Part 3

The table below provides a concise description of each MLlib component used in the pipeline, including their input/output columns and functional role.

MLlib Component	inputCol	outputCol	Explanation
RegexTokenizer	reviewText	words	Tokenizes raw text into words using a custom regex to remove unwanted characters and digits.
StopWordsRemover	words	filtered_words	Removes common stop words using a predefined list to reduce noise.
CountVectorizer	filtered_words	raw_features	Converts tokens into a sparse vector of term frequencies.
IDF	raw_features	features	Computes the inverse document frequency to weight term importance.
StringIndexer	category	category_index	Converts category labels into numeric indices for supervised learning.
ChiSqSelector	features	selected_features	Selects top N features most relevant to the label based on chi-square test.
Normalizer	selected_features	normalized_features	Applies L2 normalization to prepare features for the SVM model.
LinearSVC	normalized_features	-	Trains binary classifiers using linear SVM (used within OneVsRest).
OneVsRest	normalized_features	prediction	Extends binary SVM to multi-class classification.
CrossValidator	-	-	Performs model selection and hyperparameter tuning using 3-fold CV

Table 1: Pipeline Flow

The following diagram illustrates the end-to-end machine learning pipeline developed for multi-class text classification using Amazon product reviews. The pipeline is implemented in Apache Spark using MLlib components, and includes all essential phases - from preprocessing raw text data to model training and evaluation. This modular design enables reproducibility, scalability and efficient hyperparameter tuning across a large dataset. The pipeline is structured into three main blocks:

1. **Preprocessing Pipeline**
   Raw review text is cleaned and transformed into numerical vectors. Tokenization and stopword removal reduce noise, while term frequency (TF) and inverse document frequency (IDF) are used to compute informative text features.
2. **Feature Engineering**
   Features are further refined using chi-square feature selection and normalization. In parallel, product category labels are indexed for supervised learning compatibility.
3. **Model Training and Tuning**

A Linear Support Vector Classifier is wrapped in a One-vs-Rest strategy and is used for multi-class prediction. Cross-validation is employed to identify the best-performing model across different hyperparameter combinations. After training, the model is evaluated using F1 score to account for both precision and recall.

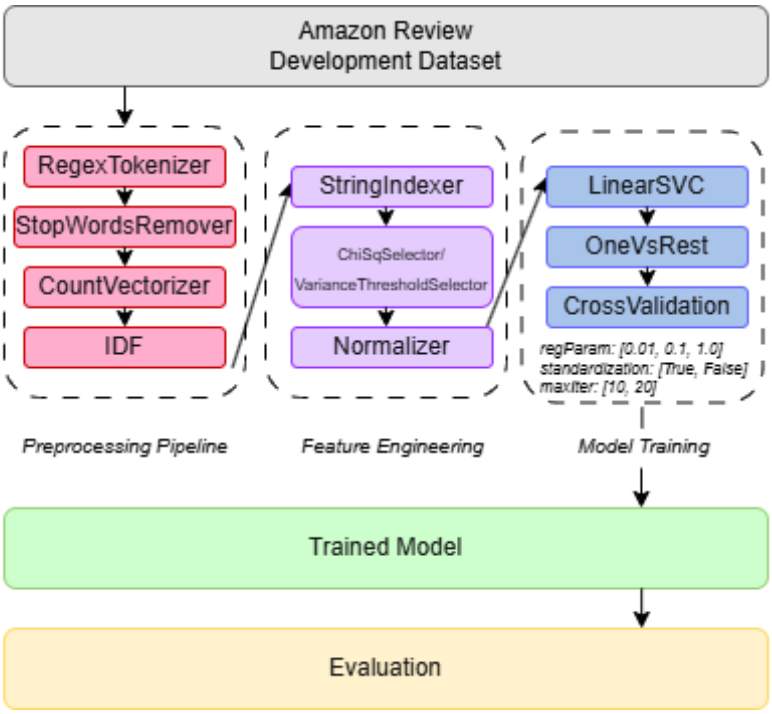


Image 1: Pipeline Graph

4. Results

Part 1

Comparing the output of Assignment 1 (when also run on the development dataset and formatting has been adjusted), we see have produced exactly the same results, which is what we would expect.

Parts 2 & 3

The table below shows the performance of the model with different hyperparameter combinations. The evaluated hyperparameters are `regParam`, `standardization`, and `maxIter`, and the evaluation score is F1. The first parameter with value of 0.01 has usually high F1 scores across most configurations, with results ranging from 0.1605 to 0.2938. Increasing `regParam` to 1.0 significantly reduces the F1 score, with the lowest scores observed for a `regParam` of 1.0 (F1 scores of 0.0424). Standardization (`True`) tends to improve performance in most cases, especially when combined with lower values of `regParam` (0.01 and 0.1). For example, with `regParam=0.1`, the F1 score is 0.3122 with standardization, compared to 0.2897 without standardization. The `maxIter` value has a moderate effect on F1 scores, with higher iterations generally yielding slightly better performance. For example, with

`regParam=0.01` and `standardization=True`, increasing `maxIter` from 10 to 12 improves the F1 score from 0.2867 to 0.2900. However, increasing iterations does not always lead to significant improvements, especially for higher `regParam` values, where the F1 scores remain relatively low regardless of the number of iterations.

regParam	standardization	maxIter	F1 score
0.01	True	10	0.2867
0.01	True	20	0.2900
0.01	False	10	0.1605
0.01	False	20	0.2938
0.1	True	10	0.3122
0.1	True	20	0.3100
0.1	False	10	0.2897
0.1	False	20	0.2973
1.0	True	10	0.1391
1.0	True	20	0.2862
1.0	False	10	0.0424
1.0	False	20	0.0424

Table 2: Results using ChiSqSelector

The table presents F1 scores for different combinations of three hyperparameters: `regParam`, `standardization`, and `maxIter`. Lower values of `regParam` (0.01 and 0.1) generally lead to better model performance, with the highest F1 score of 0.3432 observed for `regParam = 0.1`, `standardization = True`, and `maxIter = 20`. In contrast, increasing `regParam` to 1.0 consistently lowers performance, especially when `standardization` is set to `False` (e.g., F1 = 0.0631 with `maxIter = 20`). Enabling standardization (`standardization = True`) typically improves F1 scores across all `regParam` values. For instance, with `regParam = 0.1` and `maxIter = 20`, the F1 score is 0.3432 when standardization is used, compared to just 0.0732 without it. Increasing the number of iterations from 10 to 20 usually results in higher F1 scores, particularly when combined with standardization. For example, with `regParam = 0.01` and `standardization = True`, the F1 score improves from 0.3291 to 0.3404 as `maxIter` increases.

regParam	standardization	maxIter	F1 score
0.01	True	10	0.3291
0.01	True	20	0.3404
0.01	False	10	0.1937
0.01	False	20	0.2908
0.1	True	10	0.3293
0.1	True	20	0.3432
0.1	False	10	0.1935
0.1	False	20	0.0732
1.0	True	10	0.3262
1.0	True	20	0.3160
1.0	False	10	0.1935
1.0	False	20	0.0631

Table 3: Results using VarianceThresholdSelector

5. Conclusions

Part 1

As mentioned, this RDD transformation approach is only slightly higher level than the MapReducer approach in Assignment 1, so the steps are very similar but this time we can produce the same results with less code in an easy to understand Jupyter notebook.

Parts 2 & 3

The combination of `regParam=0.01`, `standardization=True`, and `maxIter=12` produces the best results, with an F1 score of 0.2900. Lower values of `regParam` tend to provide better performance, and standardization is beneficial for most settings. Increasing `maxIter` may improve results but does not have a large impact for higher `regParam` values.

Using *VarianceThresholdSelector*, the best results are achieved with lower regularization (`regParam = 0.1`), standardization enabled, and more training iterations. Disabling standardization or using a high regularization value (1.0) tends to degrade model performance substantially.

When comparing feature selection methods, *ChiSqSelector* provided more stable performance across configurations, while *VarianceThresholdSelector* showed higher variance in results - performing well under ideal settings, but degrading sharply with suboptimal hyperparameters. This suggests that *ChiSqSelector* is more robust, whereas *VarianceThresholdSelector* requires stricter control over other parameters.